

Retrieval-Augmented Generation for Scientific Code Understanding

Aaron Nobile^{*1}, Andreas Adelman^{†2}, and Mohsen Sadr^{‡2,3}

¹Department of Physics, ETH Zürich, 8093 Zürich, Switzerland

²Paul Scherrer Institute, 5232 Villigen PSI, Switzerland

³Massachusetts Institute of Technology, Cambridge, MA 02139, USA

Abstract

Large language models have become central to modern coding assistants, but state-of-the-art systems such as Claude Code or Codex [1, 2] rely on very large, cloud-hosted models with significant computational cost and data-privacy implications. This work investigates whether a useful, fully local coding agent can be built around small open-source models by shifting the computational burden away from inference. We develop a Retrieval-Augmented Generation (RAG) system for scientific code understanding that strictly separates an expensive offline ingestion stage—parsing, structural graph construction, LLM-generated entity explanations, and embedding—from a lightweight online answering stage. The system is evaluated on a 100-question benchmark spanning eleven categories over the IPPL [3, 4] scientific codebase written in C++, with answers scored by an independent frontier model as the judge. Across seven answering models, we find that model family and retrieval quality matter more than parameter count, i.e. a 9B model achieves the highest average score (0.795), outperforming both larger models within our pipeline and the same models embedded in the Claude Code retrieval architecture. The results indicate that front-loading code understanding into a reusable, codebase-specialised vector store enables small local models to deliver grounded and repository-specific answers, making the agent well suited as a privacy-preserving development tool for in-house scientific codebases.

Keywords: retrieval-augmented generation; scientific software; code understanding; local language models; privacy-preserving AI

1 Introduction

Large language models (LLMs) have become capable assistants for software development, but the strongest systems generally depend on large, cloud-hosted models. This creates practical barriers for scientific software: repositories may be private, inference can be expensive, and the relevant knowledge is often distributed across years of specialized C++ development, a challenge also observed in repository-level code generation research [5]. This work asks whether a smaller, locally executed model can answer technical questions about such a repository when the surrounding retrieval system does more of the code-understanding work in advance.

We present a retrieval-augmented system that separates expensive offline ingestion from lightweight online question answering. The offline stage parses source and documentation, constructs

^{*}nobilea@ethz.ch

[†]andreas.adelmann@psi.ch

[‡]mohsen.sadr@psi.ch; msadr@mit.edu

structural relationships, generates entity-level explanations, and embeds the resulting records. At query time, the system combines semantic retrieval, metadata reranking, query-intent routing, and structural expansion before prompting a local answering model. We evaluate the approach on 100 questions across eleven categories from the IPPL scientific C++ codebase. Our contributions are: (i) a code-aware ingestion pipeline that represents functions, files, modules, and call neighborhoods; (ii) a retrieval pipeline designed to provide concise, grounded context to small local models; and (iii) an empirical comparison of seven answering models and an agentic retrieval baseline.

2 Background

2.1 Large Language Models

A Large Language Model (LLM) is an advanced artificial intelligence system designed to process and generate sequences of text. These models are trained on massive datasets and are capable of performing a wide range of natural language processing tasks, including text generation, summarization, translation, and question answering. Modern LLMs can also understand and generate source code, making them useful tools for software development [6].

1. **Transformer Architecture** [7] - The Transformer is the core architecture behind modern LLMs. It processes text as a sequence of tokens and uses a mechanism called self-attention to determine which tokens are most relevant for understanding the current context. By stacking multiple attention layers, the model can capture complex relationships across long sequences of text.

The Transformer is a major advancement over earlier neural network architectures because it allows all tokens to be processed in parallel, significantly improving computational efficiency and scalability while preserving long-range contextual information.

2. **Parameters** - Parameters are the trainable coefficients contained within the model. They determine how information flows through the network and how strongly different patterns influence the model's predictions. During training, these parameters learn linguistic structures, semantic relationships, and statistical patterns from the training data. Modern LLMs often contain billions of such parameters.
3. **Training** - Training is the process through which the model *learns* its parameters. During this process, the model is repeatedly asked to predict missing or subsequent tokens in a text sequence. The prediction error is measured using a loss function, and optimization algorithms such as the gradient descent [8] adjust the parameters to minimize this error.

Through repeated exposure to large datasets, the model gradually learns patterns in language and develops the ability to generate coherent and contextually relevant responses.

The combination of the Transformer architecture and large-scale training has enabled language models to address a wide range of language, reasoning, and programming tasks [6, 7]. This capability has motivated specialized systems that use LLMs as their central reasoning component, commonly referred to as AI agents.

2.2 Coding Agents

One particularly successful application of LLMs is the coding agent. Rather than acting solely as a chatbot, a coding agent is able to interact with external tools, inspect source code, modify files,

execute commands, and reason about software projects. This allows the model to assist developers with tasks such as code generation, debugging, refactoring, and documentation.

The usefulness of coding agents becomes especially apparent in scientific software projects. Such codebases are often developed over many years, contain complex numerical algorithms, and require substantial domain knowledge in mathematics or physics. Understanding the structure and intent of these projects can therefore be challenging even for experienced developers.

In recent years, companies such as Anthropic [1], OpenAI [2], Google [9], and Cursor [10] have developed coding assistants including Claude Code, Codex, and Cursor. These systems rely on very large models hosted on powerful computing infrastructure. While highly capable, they require significant computational resources and are typically associated with usage costs.

The goal of this work is to investigate whether a useful coding agent can be built around a locally running open-source LLM. Since local models are generally much smaller than state-of-the-art commercial models, the surrounding agent framework must compensate for some of these limitations. By constraining the problem domain and providing structured tools and workflows, the framework aims to enable effective code understanding and generation despite the reduced model size.

2.3 Retrieval-Augmented Generation (RAG)

A major limitation of Large Language Models is that their knowledge is limited to the information contained in their training data and the context currently provided in the prompt. In the case of coding agents, this presents a challenge, as software projects often contain thousands of files and many years of development history that cannot be provided to the model at once.

Retrieval-Augmented Generation (RAG) addresses this problem by combining information retrieval with language generation [11]. Instead of relying solely on the model’s internal knowledge, a RAG system first searches an external knowledge source for information relevant to the user’s request and then provides the retrieved information as additional context to the LLM.

In the context of coding agents, the external knowledge source is typically the source code repository itself. Files, functions, classes, documentation, and comments are indexed and stored in a searchable format. When the user asks a question or requests a code modification, the agent retrieves the most relevant pieces of information from the codebase and includes them in the prompt sent to the model. This approach allows a model to work with codebases that are significantly larger than its context window and to retrieve information from the current state of the repository rather than relying exclusively on knowledge stored in its parameters [11]. These properties motivate our investigation of whether retrieval can compensate for some limitations of smaller local models.

A typical RAG pipeline consists of three main stages. First, documents are processed and converted into vector representations known as embeddings. These embeddings capture the semantic meaning of the code and documentation. Second, a retrieval system performs a similarity search to identify the most relevant documents for a given query. Finally, the retrieved information is appended to the prompt and used by the LLM to generate a response.

For coding agents, RAG serves as a bridge between the language model and the software project itself. Rather than attempting to memorize an entire codebase, the model can dynamically access the information it needs, improving both accuracy and scalability; repository-level retrieval has likewise been used to expose cross-file context to code models [5]. This principle forms the basis of the retrieval system presented here.

2.4 IPPL as a Scientific-Software Testbed

The Independent Parallel Particle Layer (IPPL) is an open-source C++ framework for developing performance-portable Eulerian, Lagrangian, and hybrid particle-mesh applications [3, 4]. Its use of MPI, Kokkos for performance-portable on-node parallelism [12], and heFFTe for distributed Fourier transforms [13] makes it a representative testbed for repository-level questions that combine software structure with high-performance-computing and numerical concepts.

3 System Design

The system is a locally executable scientific-code assistant built from open-source language-model components. Because repository-specific knowledge resides primarily in the codebase itself, the design emphasizes retrieval that supplies the answering model with structured, precise, and relevant context.

3.1 System Overview

The architecture uses *prompt enrichment*: structured excerpts from the codebase, called *chunks*, are inserted into a prompt template before it is sent to the LLM. This design cleanly separates retrieval preparation from inference. Figure 1 summarizes both stages.

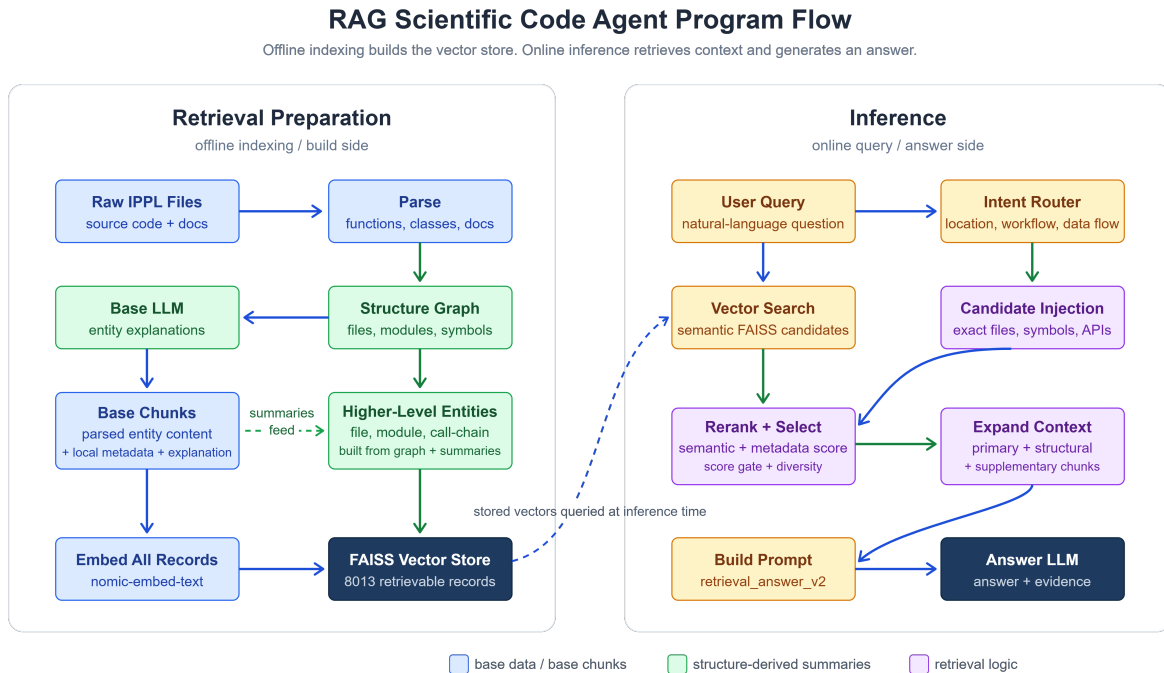


Figure 1: The flowchart of the RAG-Scientific-Code-Agent can be split into two parts: Retrieval Preparation in which we setup Structure Graphs, Base LLM explanations and Higher-Level Entities to save them as retrievable chunks in a vector store, and Inference which focuses on retrieval in a direct sense, vector search, query intent detection, candidate injection, reranking, context expansion and prompt building.

3.2 Ingestion and Retrieval Preparation (Offline)

The first stage of the pipeline consists of ingesting all relevant files from the target codebase. During this offline preprocessing step, the developed file extraction module `file_reader.py` traverses the project directory and collects source code and documentation files that may contain information relevant to subsequent retrieval tasks. The following file types are considered:

1. Source code files: `.cpp`, `.hpp`, and `.h`
2. Documentation files: `.md`, `.rst`, and `.txt`

The extracted files serve as the main building block from which relevant chunks are formed.

3.2.1 Parsing and Chunking

Since there are multiple different file types for C++ code (`.cpp`, `.hpp`, and `.h`) and documentation (`.md`, `.rst`, and `.txt`) the pipeline requires different parsers.

The main parser for `.cpp` files is `cpp_parser.py`, which parses the implementation files of the C++ Codebase. Therefore this file takes in raw `.cpp` files from the source code and determines what the system *sees* at the source level function entity. Each extracted function record contains metadata like:

```
1 {  
2     "path": ...,  
3     "file_name": ...,  
4     "base_name": ...,  
5     "source_type": ...,  
6     "entity_type": ...,  
7     "chunk_type": ...,  
8     "symbol_name": ...,  
9     "parent_symbol": ...,  
10    "class_name": ...,  
11    "return_type": ...,  
12    "parameters": ...,  
13    "leading_comment": ...,  
14    "code": ...  
15 }
```

Listing 1: Extracted metadata from function parsing

This metadata matters a lot for retrieval (section 3.3) later on, because reranking (section 3.3.6) uses fields like 'symbol_name', 'path', 'source_type', 'chunk_type', and 'parent_symbol'.

The core tool used to analyze C++ syntax is `TreeSitterParser`, built on Tree-sitter's incremental concrete-syntax-tree parser [14] and the C++ grammar distributed by `tree_sitter_languages`. Unlike regular expression matching, this approach represents C++ syntax structurally.

```
1 void ippl::initialize(int& argc, char* argv[], MPI_Comm comm) {  
2     ...  
3 }
```

Listing 2: Example of a detected function from `cpp_parser.py`

This returns:

```

1 symbol_name: initialize
2 parent_symbol: ippl
3 return_type: void
4 parameters: int& argc, char* argv[], MPI_Comm comm
5 code: full function body

```

Listing 3: Example metadata extracted during function parsing

The developed file `cpp_parser.py` contains several functions responsible for processing and organizing parsed source code entities. One such function, `_split_qualified_name(...)`, separates fully qualified names by removing the C++ scope resolution operator. For example, the function signature `void ippl::initialize()` is split into the namespace `ippl` and the function name `initialize`. This separation simplifies subsequent indexing and retrieval tasks.

Another important component of the parser is the `CommentExtractor` function. This utility associates leading comments with the corresponding parsed entities, such as functions or classes. By preserving developer-provided documentation, the parser retains contextual information that would otherwise be unavailable during retrieval.

The developed `header_parser.py` module is responsible for parsing C++ header files (`.h` and `.hpp`). Its primary purpose is to extract declarations and structural information, including function signatures, class definitions, namespaces, and other publicly exposed interfaces. In contrast, `cpp_parser.py` processes implementation files (`.cpp`) and extracts the corresponding function and method implementations. Consequently, the header parser captures the interface of the codebase, whereas the C++ parser captures its implementation.

The primary role of the header parser is to transform the contents of header files into structured representations of the codebase. The extracted entities include Namespaces, Classes, Structs, Method declarations & definitions and Qualified free-function declarations & definitions.

The Header Parsing is implemented in the `TreeSitterHeaderParser` class, it uses `TreeSitterParser` and a `CommentExtractor` just like in the `cpp parser`. Generally the Header Parser looks for:

1. `type_identifier / identifier`
2. `field_declaration_list`
3. `base_class_clause`

Then returns a record like:

```

1 {
2     "chunk_type": "class" or "struct",
3     "symbol_name": class_or_struct_name,
4     "parent_symbol": class_or_struct_name,
5     "return_type": "class" or "struct",
6     "parameters": inheritance_clause,
7     "code": full class/struct source,
8 }

```

Listing 4: Record produced by the header parser

3.2.2 Documentation chunking & parsing

For documentation parsing, all documentation files within the codebase are processed by `doc_parser.py`. The parser identifies and extracts logical sections from Markdown, reStructuredText, and plain text documents.

These sections do not yet correspond to the final entries stored in the vector database. Instead, they are passed to the `DocChunker`, which converts them into structured chunk records containing metadata such as `symbol_name`, `chunk_type`, `code_block`, and `entity_level`. To ensure that individual chunks remain within a predefined size limit, the `DocChunker` further subdivides sections whose content exceeds the configured `max_chunk_size`.

Each resulting chunk is represented as a retrievable record of the form:

```
1 {  
2   "source_type": "documentation",  
3   "symbol_name": section_title,  
4   "chunk_type": section_type,  
5   "language": "markdown" / "rst" / "text",  
6   "section_path": ...,  
7   "code": chunk_content,  
8   "entity_type": section_type,  
9   "entity_level": "documentation_section_level",  
10 }
```

Listing 5: Documentation chunk record

Since the chunker enforces a maximum chunk size, a single documentation section may be divided into multiple chunks. In such cases, the corresponding section title is extended with a numerical suffix (e.g., `section_title_1`, `section_title_2`, ...) to preserve the relationship between the generated chunks and their original section.

3.2.3 Call and Dependency Graph Building

Large Code bases are often time concatenated with multiple functions, files and folders. In addition, large scientific simulations also contain various call-chains. The Language model needs to retrieve these correctly in order to deliver a good explanation. `project_structure_builder.py` inside `src/structure` tries to solve this issue by building several "structure graphs" after parsing. `ProjectStructureBuilder` combines them into one `project_structure`. The main output is stored inside `embeddings/project_structure/project_structure.json`.

3.2.4 Higher Entity Building

As of now retrieval preparation only creates chunks based on code structures hence one chunk is a entire function, a class, a namespace etc... This function-level chunking technique is very common and ensures that upon retrieval there is not much noise inside the retrieval context. However what happens if the user query asks about an entire Module?

```
1 What files are inside the FFT module in the source code?
```

Listing 6: Example query for which a summary of the FFT module should be retrieved

A query like this faces a problem, even if all the functions inside the files of the FFT module would get retrieved, which is hardly possible, the context-size would flood the LLM with noise and deliver a wrong result. A solution however to this problem is building a specific "Higher Entity Chunk" specifically for the module level in this case the FFT module. This is exactly what the

program does. Using the project structure alongside the file-contents of the FFT module the program creates a structured list of contents which get sent to a LLM to create a specific Module level chunk. It is reasonable that this steps also occurs also on the file-level stage.

In `runtime_settings.json`, the user can choose different entity building strategies. There are three settings, which can be adjusted:

Setting	Meaning
<code>file_level_entity_strategy</code>	How file-level summary entities are created.
<code>module_level_entity_strategy</code>	How module/folder-level summary entities are created.
<code>call_chain_entity_strategy</code>	How call-chain/workflow entities are created.

Table 1: Entity-building strategy settings, enlisted in the runtime-settings.

`unified_whole_file_with_fallback_v1` creates one file-level entity per file. It uses parsed symbols and their explanations when available. If no symbols were parsed, it falls back to visible whole-file/raw content evidence.

`module_entities_from_descendant_files_v1` creates module-level entities from the files inside a module/folder. It reads lower file summaries and structural relationships to describe what the module is responsible for.

`direct_call_neighborhood_v1` creates call-chain entities around callable symbols. It uses direct incoming and outgoing call relationships, plus symbol/file/module summaries, to explain a local workflow neighborhood.

Therefore the project structure has three main node-like lists: "symbols" (functions, classes, structs,...), "files" and "modules" (folders and subfolders)

3.2.5 Embedding and Vector Store

The embedder, implemented in `embedder.py`, serves as the interface between the FAISS vector store [15] and the retrievable content chunks. It has two primary responsibilities:

1. Embedding the chunked content.
2. Embedding the user query.

The overall purpose of the embedder is to transform both the stored content and the user's question into vector representations such that semantically related content yields a high similarity score during retrieval. When content chunks are embedded, the pipeline incorporates their parsed information and associated metadata into a structured representation. This process is implemented in `_build_chunk_embedding_prompt` and follows a predefined format.

```

1 File: ...
2 Symbol: ...
3 Chunk Type: ...
4 Parent Symbol: ...
5 Section Path: ...
6 Generated Explanation:
7 ...
8 Code:
9 ...

```

Listing 7: Content chunk structure generated inside `embedder.py`

This design is a distinguishing feature of the pipeline, as both semantic explanations and structural metadata become part of the final vector representation. In addition, the embedder applies a `prompt_limit` to the generated embedding prompt. During fallback retries, the fields `generated_explanation` and `code` may be truncated to satisfy the context length constraints of the embedding model. Consequently, if an embedding attempt fails due to excessive prompt length, a second attempt is performed using a shortened version of the chunk.

Since the embedder is also responsible for embedding user queries, it contains initial mechanisms for query intent detection. Before vectorizing a user query, the embedder searches for file name patterns using a regular expression.

```
1 self.file_extension_pattern = re.compile(
2 r"\b[A-Za-z0-9_-]+(?:\.cpp|hpp|h|md|rst|txt)\b",
3 re.IGNORECASE,
4 )
```

Listing 8: Regular expression used to detect file names such as `Ippl.cpp` or `BareField.h` in a user’s query

If a file name is detected in the user query, the embedder explicitly incorporates this information into the query embedding prompt generated by `_build_query_embedding_prompt`. The objective is to structure the query in a similar manner to the embedded content chunks, thereby increasing the likelihood that relevant chunks are retrieved.

It is important to note that this prompt structure is only used for generating the query embedding. The original user query sent to the LLM remains unchanged.

```
1 def _build_query_embedding_prompt(self, text):
2     file_names = self._extract_file_names(text)
3     file_name = ", ".join(file_names)
4     intent = (
5     "find relation across FILES which is asked in QUESTION"
6     if len(file_names) > 1
7     else self._infer_intent(text)
8     )
9     chunk_types = self._find_chunk_type(text)
10    chunk_type = ", ".join(chunk_types) if chunk_types else "any"
11    detected_code = self._extract_detected_code(text)
12
13    """
14    sections = []
15    if file_name:
16        sections.append(f"File: {file_name}")
17    if chunk_type:
18        sections.append(f"Chunk Type: {chunk_type}")
19    if intent:
20        sections.append(f"Intent: {intent}")
21    sections.append(f"Question:\n{text}")
22    if detected_code:
23        sections.append(f"Code:\n{detected_code}")
24    """
```

Listing 9: Query embedding prompt template implemented in `embedder.py`

Depending on the content of the user query, information such as file names, chunk types, inferred intent, and detected code fragments is extracted and incorporated into the embedding prompt. This allows the query embedding to more closely resemble the structure of the indexed content chunks, improving the effectiveness of dense retrieval before reranking is applied.

3.2.6 Vector Store

The vector store is the "library" in which all vectorized content chunks get stored and where `retriever.py` fetches all Top-k chunks from it. The vector store gets built by the python script `vector_store.py`. Since generating embeddings and building the vector store is computationally demanding the vector store relies on a persisted manifest which signals the pipeline to not rebuild the vector store if unchanged.

```
1 embedding_backend: ollama
2 embedding_model: nomic-embed-text
3 dimension: 768
4 metadata_count: 8013
5 vector_count: 8013
6 max_chunk_size: 400
```

Listing 10: Persisted vector-store manifest used to identify reusable artifacts

For both searching and storing vectors the pipeline uses FAISS [15]. When the `VectorStore` class is initialized, its FAISS index is configured for exact L2-distance search. All embeddings are converted to 32-bit floating-point values before insertion.

Upon entering a specific user question the `search` function inside `vector_store.py` uses the embedded query vector to find top-k closest chunks. Each top-k retrieved chunk will be stored inside results array alongside the chunk content and the determined distance.

```
1 def search(self, query\_vector, k=5):
2     #embeded prompt of user = query\_vector -> turn to float32
3     #k...amount of neigheerst search results returned.
4     query_vector = np.array([query\_vector]).astype("float32")
5     #distances = vector L2 distances away from the vector (rising)
6
7     distances, indices = self.index.search(query\_vector, k)
8     results = []
9     for distance, index in zip(distances[0], indices[0]):
10         if index < 0 or index >= len(self.metadata):
11             continue
12         results.append(
13             {
14                 "chunk": self.metadata[index],
15                 "distance": float(distance),
16             }
17         )
```

Listing 11: Search in `vector_store.py` returns the top-k chunks and their distances from the query vector

All previously shown implementations follow Attention based dense retrieval search only, however hybrid retrieval is important since dense retrieval can miss direct keyword search. For this the vector store appends the retrieved chunk list by using three functions: `get_chunks_by_filenames`, `get_chunks_by_symbols` and `get_chunks_by_modulenames`.

The vector store provides several exact-retrieval helpers in addition to normal semantic vector search. `get_chunks_by_filenames` retrieves file-level entities for explicitly mentioned files. It distinguishes between a full file name, such as `BareField.hpp`, and a base name without extension, such as `BareField`. If only the base name is mentioned, all existing file-level entities with that base name are injected, e.g. `BareField.h` and `BareField.hpp`, with distance 0.0. If a full file name is mentioned, the exact file receives distance 0.0, while sibling files with the same base name but different extensions are also added using their normal vector distance.

`get_chunks_by_symbols` retrieves function-level chunks whose symbol name exactly matches symbols detected in the query. To improve precision for queries that mention both a function and its containing file or class, the method checks whether the matched chunk's `base_name`, file stem, or full `file_name` also appears in the query-derived symbols. Such contextual matches receive distance 0.0, while broader file-subject matches are kept slightly lower priority. Only the top two symbol candidates are returned.

`get_chunks_by_modulenames` retrieves module-level entities by exact module name. It is intended to provide a direct retrieval path for questions that explicitly mention a module or subsystem, returning matching module-level chunks with distance 0.0.

`search_in_filenames` performs a filename-scoped semantic search. Unlike exact file injection, it computes vector distance between the query and chunks inside the given files. It is restricted to `function_level` chunks and returns the top three closest matches, which helps retrieve the most relevant functions or methods from already identified files.

The whole vector store in the end gets saved under `embeddings/vector_store` therefore without changing the manifest the pipeline calls up the already stored `vector_store` when retrieving chunks.

3.3 Retrieval

Retrieval is the component of the system responsible for selecting the most relevant pieces of information from the indexed codebase before an answer is generated. Instead of sending the entire project to the language model, the retrieval pipeline searches over embedded code chunks, structural metadata, symbol names, file paths, and higher-level entity descriptions to identify a compact context that is likely to contain the answer. This step is essential because large codebases exceed the context window of an LLM and contain many unrelated files. By ranking and filtering candidate chunks according to semantic similarity, lexical matches, and entity-level relevance, retrieval acts as the bridge between the user's question and the parts of the repository that the model should reason about. A strong retrieval stage therefore directly improves answer quality: it reduces noise, surfaces the right implementation details, and makes the final response more grounded in the actual source code.

3.3.1 Chunk Classification

After the candidate chunks have been retrieved, they are categorized into three groups based on their role within the retrieval pipeline:

1. **primary:** main chunks selected directly from the candidate pool after semantic search, exact filename/symbol injection, re-ranking, score gating and diversification. They are the core answer evidence
2. **structural_expansion:** chunks added after primary retrieval by `structural_expander.py`. They are structurally related to the primary chunks e.g. file-level chunk can expand module-level chunk; function-level chunk can expand call-chain chunk etc.
3. **supplementary:** chunks retrieved from files referenced by the primary chunks. The retriever collects `referenced_files` from **primary chunks**, builds a supplementary query and searches inside those referenced files using `search_in_filenames(...)`

3.3.2 Retriever

The central component that orchestrates retrieval is `retriever.py`. Its primary task is to decide which retrieved content chunks are ultimately passed to the LLM and which candidates are filtered out. This functionality overlaps conceptually with parts of the vector store described in subsection 3.2.6, but the two components operate at different stages of the retrieval process. The functions in `vector_store.py` are responsible for storing chunks and computing their initial relevance to a query, including adjustments to the retrieval distance for exact file-name or symbol-name matches. In contrast, `retriever.py` works with the ranked candidates returned by the vector store and decides how these candidates should be combined, filtered, and balanced before being sent to the LLM.

In this sense, the vector store answers the question: “Which chunks exist, and how close are they to the query?” The retriever answers the follow-up question: “Which mixture of chunks should the LLM actually see?” This makes the retriever the final selection layer between similarity search and answer generation.

For choosing the final candidates, the retriever uses several configurable parameters during the initialization of the `Retriever` class. In particular, it defines five adjustable settings that control the selection of primary chunks.

Setting	Value	Description
<code>PRIMARY_SCORE_RELATIVE_FLOOR</code>	0.4	Controls how far below the best reranked candidate may fall before it is considered weak. Used in <code>_select_primary_candidates(...)</code>
<code>PRIMARY_SCORE_GAP_THRESHOLD</code>	8.0	Controls how large a score drop between two consecutive accepted candidates must be before the retriever treats it as a quality cliff. The value states the difference; as soon as <code>"diff" ≥ PRIMARY_SCORE_GAP_THRESHOLD</code> the selection stops if the candidate is also below the relative floor.
<code>PRIMARY_SCORE_ABSOLUTE_FLOOR</code>	1.25	Defines minimum absolute score threshold. Below this threshold → very weak candidate.
<code>PRIMARY_SAME_SYMBOL_CAP</code>	1	Limits how many primary chunks with same symbol group (file path + symbol name) we can retrieve.
<code>PRIMARY_SAME_FILE_CAP</code>	2	Limits how many non-file-level primary chunks with the same filename we can retrieve.
<code>DOMINANT_FILE_RATIO</code>	0.75	Detects when the selected primary candidates are mostly from one file, the value describes a threshold value for <code>dominant_count / total_count</code> . Meaning: exceeding the threshold has too many chunks from one file only.

Table 2: Constants to control which final primary chunks the retriever chooses. They are mainly used to avoid weak or repetitive context and can be adjusted manually in the config `runtime_settings.json`.

These Parameters get used by the main function of the retrieval pipeline: `retrieve_with_diagnostics(self, query, k=5) ...`. The function takes the user query and returns:

```

1  {
2    "chunks": final_retrieved_chunks,
3    "diagnostics": debug_information
4  }
```

So it does both retrieve context for the answer LLM and record why those chunks were selected. In addition the function extracts query signals such as:

```
1     exact_filenames
2     exact_symbols
3     api_bearing_terms
4     data_flow_direction
5     data_flow_terms
6     comparison_subjects
```

Next it calls `QueryIntentRouter` to classify the user query, into one of the following categories:

```
1     api_usage
2     location_lookup
3     data_flow
4     comparison
5     symbol_explanation
6     file_purpose
7     module_overview
8     workflow_explanation
9     default
```

After extracting both the categoric terms with the reranker and the query intent with the `QueryIntentRouter`, the function calls up the implemented FAISS vector search implemented in `vector_store.py`. The `candidate_count` is determined both by the selected candidate number `k` and the `self.candidate_k=20` set, which can be adjusted in the config. The semantic candidates are chosen from the search function inside the vector store file. Then exact filename/symbol targets and candidates are acquired by the corresponding functions inside the `vector_store`. For comparison intent the retriever uses a specific `self._retrieve_comparison_candidates(...)` function to prevent flooding the candidate pool with only one side of mentioned exact key-name hits.

Candidate source	Purpose	Active when
Semantic candidates	Top- k vector-search results from the full vector store. This is the broad semantic retrieval baseline.	Always
Exact filename candidates	Injects file-level chunks for explicitly mentioned filenames or basename matches, e.g. <code>BareField</code> matching <code>BareField.h</code> and <code>BareField.hpp</code> .	Always
Exact symbol candidates	Injects function-level chunks whose symbol name exactly matches query symbols, e.g. <code>getAllocated</code> .	Non-comparison queries
Subject file candidates	Injects file-level chunks whose filename stem or basename matches a query subject. This anchors queries around files/classes mentioned without extension.	Non-comparison queries
Comparison candidates	Injects balanced anchors for each compared subject, so both entities in a comparison receive evidence.	Comparison queries
Literal API candidates	Scans chunk text for concrete API calls such as initialization/finalization calls and injects exact call-site evidence.	API/location queries
Target-aligned candidates	Injects chunks matching an explicitly requested entity level, such as file-level, module-level, function-level, or call-chain-level.	When entity target is detected

Table 3: Candidate pool sources used by the retrieval pipeline before reranking and primary chunk selection.

After enlisting all possible candidates separately `_merge_candidates(...)` concatenates all candidate sources. It prioritizes injected candidates by order, semantic vector candidates last. Then removes all duplicate chunks. The actual scoring/ranking happens afterward in the reranker. After re-ranking the pipeline the candidates get refined a final time. `_select_primary_candidates` removes weak reranked primary candidates. `_refine_primary_candidates` applies query-intent logic and diversify and `_ensure_exact_filename_chunks` implements final exact-match ordering safeguard.

3.3.3 Test/Build Noise Sorting

Since the vector store contains build and test files in addition to implementation files from the source code, the program needs to distinguish between these categories during retrieval. Test and build files are usually not part of the core implementation; instead, they often demonstrate, validate, or configure functionality that appears elsewhere in the codebase. The constant `TEST_MODULE_SCOPES` defines which module scopes the retriever treats as test or build noise, unless the user explicitly asks about tests or build-related files.

```

1  TEST_MODULE_SCOPES = frozenset({"test", "tests", "unit_test", "unit_tests"})
2  BUILD_FILE_NAMES = frozenset({"cmakelists.txt"})
3  TEST_QUERY_KEYWORDS = (
4      "test",
5      "tests",
6      "unit test",
7      "unit_test",
8      "unittest",

```

```

9     "cmake",
10    "cmakelists",
11    "build",
12    "integration test",
13 )

```

This constant lists the module scopes that are considered test- or build-related. A chunk is treated as belonging to this category if its metadata contains a module scope such as:

```

1 module_scope = "test, tests, ..."

```

or:

```

1 module_scope = "unit_tests, cmake, cmakelists, ..."

```

The corresponding filtering logic can be summarized as follows:

```

1 The retrieved chunk comes from a test or build file
2 AND the query does not mention tests or build configuration
3 THEN remove that chunk from the final context.

```

However, if the user explicitly asks a test- or build-related question, for example:

```

1 Where is this tested?
2 How is the build configured?
3 What does CMakeLists.txt do?

```

then chunks whose module scope is listed in `TEST_MODULE_SCOPES` remain eligible for retrieval. This prevents test and build files from polluting the context of general implementation-focused answers, while still allowing them to be retrieved when they are directly relevant to the query.

3.3.4 Structural Expansion

Structural expansion is a method used in the pipeline of the retriever by calling

```

1     structural_result = self.structural_expander.expand(
2     query,
3     primary_chunks,
4     mode=intent_result["structural_mode"],
5 )

```

Structural expansion is applied after the primary retrieval step. First, the retriever collects candidates, reranks them, selects the primary chunks, and applies exact-match safeguards. Only then does it call the structural expander on the selected primary chunks. The expansion mode is chosen by the query intent, for example API-usage queries use a narrower expansion profile, while data-flow and workflow queries use a broader profile. The expander does not perform a new semantic search; instead, it follows structural metadata relationships that were built during ingestion. Depending on the entity level of a primary chunk, it can add related file-level, module-level, call-chain-level, or symbol-level chunks. For example, a selected function-level chunk can be expanded to its file-level summary or module-level summary if the active profile allows it. Each expansion profile limits which relationships are followed and how many additional chunks may be added. Thus, structural expansion enriches trusted primary evidence with nearby structural context while keeping the amount of added context bounded.

The number of added chunks is capped by the active profile:

```

1 mode_api_usage -> max 3 expansions
2 mode_1_minimal -> max 2 expansions
3 mode_2_balanced -> max 3 expansions
4 mode_3_broad -> max 4 expansions

```

The conditions to structurally expand are:

```

1 1. There must be primary_chunks.
2 2. The QueryIntentRouter must assign a structural_mode.
3 3. The StructuralExpander must find related chunks in its indexes.
4 4. The expansion profile must allow that kind of relationship.
5 5. The related chunk must not already be one of the primary chunks.
6 6. The max_total_expansions limit must not be reached.

```

3.3.5 Query Intent Routing

Query intent routing describes the process of identifying the purpose of a user query before the final retrieval context is assembled. This is an important part of the retrieval pipeline because the amount and type of structural expansion should depend on what the user is asking for. For example, a location lookup should retrieve a narrower context than a workflow or data-flow question, which usually requires broader surrounding information.

The current implementation in `query_intent_router.py` categorizes queries into the following intent classes:

Query intent	Structural expansion mode
api_usage	mode_api_usage
location_lookup	mode_1_minimal
data_flow	mode_3_broad
comparison	mode_2_balanced
symbol_explanation	mode_2_balanced
file_purpose	mode_2_balanced
module_overview	mode_2_balanced
workflow_explanation	mode_3_broad
default	mode_2_balanced

Table 4: Query intent categories and their corresponding structural expansion modes.

In addition to assigning an intent, `query_intent_router.py` also detects whether the query explicitly targets a certain entity level. Keywords such as “file”, “module”, “call chain”, “function”, and “documentation section” are translated into concrete retrieval preferences. These preferences tell the retriever which entity levels or chunk types should be preferred when ranking the final context.

The constant `SYMBOL_LEVEL_CHUNK_HINTS` follows the same principle for symbol-level queries. It maps query terms such as “class”, “struct”, “method”, “function”, “routine”, and “namespace” to more specific chunk types. For example, “function” is mapped to `function_definition` and `method_definition`, while “namespace” is mapped to `namespace`. These hints are only applied when the query targets symbol-level information.

The `route(...)` function is responsible for deciding which kind of question the user asked and which structural expansion mode and retrieval preferences should be used. The function proceeds as follows:

1. Detect whether the query explicitly asks for a specific entity type using `_detect_entity_target(...)`.
2. Detect the query intent, for example API usage, location lookup, data flow, comparison, workflow explanation, or module overview.
3. Return the detected intent, structural expansion mode, routing reasons, entity target, and retrieval preferences.

3.3.6 Metadata Reranker

The file `reranker.py` implements the metadata reranker used inside the retrieval pipeline. It takes the candidate chunks returned by the vector store and reranks them before the structural expansion step. As described in subsection 3.2.6, the vector store returns an L2 distance between the query embedding and each retrieved chunk embedding. The reranker converts this distance into a semantic score and combines it with metadata-based signals that describe how well a chunk matches the query on the level of file names, symbols, namespaces, source types, API calls, data-flow terms, and requested entity types.

```
1 semantic_score = 1.0 / (1.0 + max(distance, 0.0))
2
3 metadata_score = (
4     lexical_metadata_weight * lexical_metadata_score
5     + entity_target_weight * entity_target_score
6 )
7
8 combined_score = semantic_score + metadata_score
```

Listing

12:

Scoring formula for the reranked candidate score. The parameters `lexical_metadata_weight` and `entity_target_weight` are configurable in `runtime_settings.json`.

The `lexical_metadata_score` captures direct textual and structural overlap between the query and a candidate chunk. It rewards exact filename matches, exact symbol matches, namespaced symbol matches, namespace matches, and overlap with file paths or symbol names. It also considers whether the query explicitly asks for a header, source, or documentation file and whether the chunk has the corresponding `source_type`.

The `entity_target_score` captures whether the chunk matches the type of entity requested by the query intent router. For example, if the user asks for a module, module-level chunks are boosted; if the user asks for a file, file-level chunks are preferred. This prevents lexically related but structurally wrong chunks from dominating the final ranking.

The reranker also contains several query-specific boosts. For API or location queries, it extracts API-bearing terms such as `Kokkos::initialize` and rewards chunks that contain the corresponding call sites. For data-flow questions, it detects directions such as grid-to-particles or particles-to-grid and boosts chunks containing terms such as `gather`, `scatter`, `interpolation`, or related particle/grid terminology. For comparison questions, it extracts the compared subjects and rewards chunks whose metadata or text matches one of those subjects.

In addition, `reranker.py` includes safeguards against misleading matches. Very low-signal symbol names such as `i`, `j`, or `x` are penalized, because they frequently occur as local variables and are not useful retrieval anchors. Test-scope chunks are also penalized for non-test location queries, so that test files containing the same API calls do not outrank the actual implementation. Finally,

the reranker can return diagnostics, including the extracted query tokens, exact filenames, exact symbols, matched API terms, entity preferences, and the full list of reranked candidates.

3.4 LLMAgent

LLMAgent does a conservative cleanup after retrieval in `_filter_final_chunks()`. It does not rerank chunks again but focuses on filtering chunks which are too weak to show to the LLM, and asks if there is already better evidence from the same location.

A chunk gets a final-context quality score in `llm_agent.py`. It gets positive points for:

1. having a generated explanation
2. having a leading comment
3. having include paths or referenced files
4. being high-level context: `file_level`, `module_level`, or `call_chain_level`
5. having enough real code content

Negative points are given to chunks if:

1. `generated_explanation_status == "skipped_low_information"`
2. being a trivial declaration, including empty or tiny declarations like `class X;`

Then the drop logic removes a chunk if all of the following are true:

1. Its quality score is low, currently ≤ 0
2. It is either marked `skipped_low_information` or is a trivial declaration
3. There is another chunk from the same local context
4. That other chunk has a good enough score, at least 2.
5. That other chunk beats it by at least 3 points.

So the practical effect of the LLMAgent is: if retrieval returns both a useless `class FFT ;` declaration and a richer FFT file-level or method chunk, the agent may drop the empty declaration before building the prompt.

In general `LLMAgent.py` orchestrates the connection between the reranked retrieved context and the LLM, after filtering it calls up `build_context()` to turn selected chunks into a text context string. This assures that from all context in the chunk a systematic and structural text is formed. This string gets inserted in the prompt and sent to the LLM.

3.5 Prompting and Inference

The project uses local LLM calls in several different stages of the pipeline. Therefore, it does not rely on a single prompt template, but defines separate prompt modes for different inference tasks. The first LLM calls already occur during retrieval preparation, where parsed entities are enriched with generated explanations. For symbol-level chunks, this is handled by the `EntityExplanationGenerator`; for higher-level entities, the `file_level_entity_builder.py`, `module_level_entity_builder.py`,

and `call_chain_entity_builder.py` modules use dedicated prompt templates together with `LLMWrapper` to generate retrieval-oriented entity explanations.

The prompt templates are defined in `prompt_templates.py`. Each prompt mode is designed for a different type of context: local code entities, file summaries, module summaries, call-chain neighborhoods, or the final RAG answer. The active prompt modes are configured in `runtime_settings.json`, while the complete prompt templates are listed in Appendix B.

Prompt mode	Entity / use case	Prompting technique
<code>general</code>	One parsed code entity or documentation section	Local entity explanation with a fixed structured schema
<code>file_level</code>	Whole file with parsed symbols	Retrieval-friendly synthesis from structured file facts
<code>file_level_fallback</code>	Whole file without parsed symbols	Conservative fallback based on raw file evidence
<code>module_level</code>	Folder or module summary	Aggregation of file-level summaries and module responsibilities
<code>call_chain</code>	Callable plus incoming and outgoing calls	Local call-graph neighborhood explanation
<code>retrieval_answer</code>	Final RAG answer, older style	Broad answer synthesis with several reasoning sections
<code>retrieval_answer_v2</code>	Final RAG answer, evaluation style	Strict grounded question answering with constrained output

Table 5: Prompt modes and their associated prompting techniques.

An important implementation detail is that prompt templates are part of the runtime manifest. The system computes prompt signatures for the active templates, so changing a prompt can invalidate previously generated artifacts and trigger a rebuild. This ensures that stored explanations remain consistent with the prompt version that produced them.

During final inference, `LLMagent` formats the retrieved chunks into a structured context before calling the answering model. Each chunk is represented with metadata such as entity level, symbol name, path, namespace, generated explanation, references, and structural relationships. The final answer prompt therefore receives both semantic explanations and code-structure signals, while the prompt itself constrains the model to answer only from the retrieved context.

4 Implementation and Runtime Configuration

Since the architecture of this RAG-Scientific-Code-Agent separates retrieval preparation from the inference completely. It behaves fundamentally different than approaches in which retrieval preparation and inference are tightly coupled at query time. The main idea behind the architecture decision is to move as much computationally expensive work as possible into an offline pre-processing stage, performed once, when initializing the agent to a new codebase.

This is useful since codebase agents, are mainly used repeatedly within the same project environment, even though established models are designed to be adaptable to different codebases. In the proposed architecture, the system first ingests the target codebase and generates LLM-friendly retrieval context offline, using comparatively strong computational resources. The resulting vector store can then be reused during inference. As a result, the online answering stage can rely on smaller

or more efficient models, because much of the code understanding has already been encoded into generated explanations, structural metadata, and retrieval-ready entities.

This section discusses the advantages and disadvantages of this architectural decision and documents the intended way of using, running and adapting the program to the ippl or other codebase environments.

4.1 Configuration and Runtime Setup

The configurable architecture of the software allows several important parameters for ingestion, embedding, prompting, retrieval, and model selection to be adjusted without changing the source code directly. These parameters are collected in `config/runtime_settings.json`. This file therefore acts as the central runtime configuration of the pipeline and defines how the codebase is parsed, which models are used, how explanations are generated, and how retrieved chunks are ranked before they are passed to the final answering model.

For embedding, the active backend is `ollama`, using `nomic-embed-text` [16] as the embedding model. The embedder also supports a sentence-transformer backend through the `sentence_transformer` option, with `BAAI/bge-code-v1` listed as the configured sentence-transformer model. In the current evaluation setup, however, embeddings are generated locally through Ollama.

```
1  "embedding": {
2    "backend": "ollama",
3    "ollama_model": "nomic-embed-text",
4    "sentence_transformer_model": "BAAI/bge-code-v1"
5  },
```

The configuration also defines the model roles used throughout the pipeline. The chunk-level explanation model is set to `qwen2.5-coder:32b`, while the file-level, module-level, call-chain, and final answer models use `qwen2.5-coder:32b-instruct-q4_K_M`. This separates the model used for symbol-level code explanations from the instruction-tuned model used for higher-level summaries and final question answering.

```
1  "models": {
2    "answer_model": "qwen2.5-coder:32b-instruct-q4_K_M",
3    "chunk_explanation_model": "qwen2.5-coder:32b",
4    "file_level_model": "qwen2.5-coder:32b-instruct-q4_K_M",
5    "module_level_model": "qwen2.5-coder:32b-instruct-q4_K_M",
6    "call_chain_model": "qwen2.5-coder:32b-instruct-q4_K_M"
7  },
```

Prompt behavior is also configured in `runtime_settings.json`. The final answering stage uses `retrieval_answer_v2`, while the preprocessing stages use dedicated prompt modes for chunk explanations, file-level summaries, module-level summaries, and call-chain summaries. This makes it possible to change the prompting strategy for one stage without affecting the others.

```
1  "prompts": {
2    "answer_prompt_mode": "retrieval_answer_v2",
3    "chunk_explanation_prompt_mode": "general",
4    "file_level_prompt_mode": "file_level",
5    "file_level_fallback_prompt_mode": "file_level_fallback",
6    "module_level_prompt_mode": "module_level",
7    "call_chain_prompt_mode": "call_chain"
8  },
```

The retrieval section of the configuration controls how aggressively candidates are selected and reranked. Parameters such as `candidate_k`, `supplementary_k`, `primary_same_symbol_cap`, and `primary_same_file_cap` influence how many chunks are considered and how much duplicate context from the same file or symbol is allowed. The weights `lexical_metadata_weight` and `entity_target_weight` control the importance of metadata overlap and query-intent preferences inside the reranking score.

```

1  "retrieval": {
2    "candidate_k": 20,
3    "supplementary_k": 3,
4    "supplementary_candidate_k": 10,
5    "primary_score_relative_floor": 0.4,
6    "primary_score_gap_threshold": 8.0,
7    "primary_score_absolute_floor": 1.25,
8    "primary_same_symbol_cap": 1,
9    "primary_same_file_cap": 2,
10   "dominant_file_ratio": 0.75,
11   "lexical_metadata_weight": 0.4,
12   "entity_target_weight": 0.7,
13   "debug_enabled_by_default": false
14 }

```

To adapt the system to a different codebase, the most important setting is `raw_data_path`, which should point to the new source-code directory. Depending on the target project, the ingestion settings may also need to be adjusted, for example by changing the preferred parser, the maximum chunk size, or the entity types for which explanations should be generated. The model settings can be changed if different local models are available, and the retrieval parameters can be tuned if the new codebase requires broader or narrower context selection. After such changes, the vector store should be rebuilt with a clean rebuild so that the stored embeddings, generated explanations, and manifest match the new configuration.

4.2 Execution on the Cluster

When initializing the project for IPPL or for another codebase, the vector store has to be built from scratch. This initial build is the most computationally expensive stage of the pipeline, because the system parses the codebase, generates LLM-based entity explanations, computes embeddings, and stores the resulting chunks in the vector store. For this reason, the preprocessing step should be executed on a compute node with sufficient GPU resources.

In this project, the preprocessing job is submitted through SLURM using the command:

```
1 FORCE_CLEAN_REBUILD=1 sbatch job.sh
```

The flag `FORCE_CLEAN_REBUILD=1` ensures that previously generated artifacts are removed and that the vector store is rebuilt from scratch. This is useful when changing important configuration parameters, such as the selected models, prompt templates, parsing strategy, retrieval settings, or the underlying source code that should be indexed.

The vector store used in this project was generated on the `gmerlin6` cluster, using the `gwendolen` partition. The submitted job requests one GPU and was executed on nodes equipped with NVIDIA A100 GPUs with 40 GB of GPU memory. These resources are especially important during the explanation-generation stage, where local LLM inference is repeatedly applied to parsed code entities. Once the vector store has been generated, the expensive preprocessing artifacts can be reused, and later inference runs no longer require rebuilding the full index unless the configuration or input codebase changes.

4.3 Architectural Advantages and Limitations

The architecture of the RAG-Scientific-Code-Agent differs from general-purpose coding agents such as Codex [2] or Claude Code [1]. Codex and Claude Code are designed as interactive agentic coding systems: they can inspect a repository, reason over the current task, edit files, run commands, execute tests, and iterate based on the results. In contrast, the system developed in this project focuses on retrieval-augmented code understanding. Its main architectural decision is to separate expensive retrieval preparation from later inference. The codebase is parsed, enriched with generated explanations, embedded, and stored in a vector store before the user asks questions.

This design has several advantages. First, expensive work is moved into an offline preprocessing stage. LLM-based chunk, file, module, and call-chain explanations only have to be generated once for a given codebase version and configuration. During inference, the system can reuse this prepared retrieval context instead of repeatedly asking the model to rediscover the structure of the repository. Second, the generated vector store is specialized for the target codebase. This is especially useful for scientific C++ projects, where relevant information is often distributed across templates, headers, implementation files, namespaces, and build or test directories. Third, the architecture improves reproducibility: the runtime settings, prompt modes, model names, and generated artifacts can be stored and compared across evaluation runs. Finally, because much of the code understanding is already encoded into retrieval-ready explanations and metadata, the final answering step can be performed with a smaller or more efficient local model than would otherwise be required.

However, this architecture also has disadvantages. The initial preprocessing step is computationally expensive and can require a powerful GPU node, especially when explanations are generated for many code entities. The vector store can also become stale: if the source code, parser, prompt templates, or model configuration changes, the index may need to be rebuilt. In addition, the system depends strongly on the quality of parsing, chunking, embedding, reranking, and structural expansion. If an entity is not extracted correctly or not retrieved, the final LLM cannot reason about it. Compared with agentic systems such as Codex [2] or Claude Code [1], the system is also less autonomous. It does not directly modify code, run tests, inspect errors, or iteratively verify its own answer by executing commands.

The comparison therefore shows a trade-off between specialization and autonomy. Codex and Claude Code are more flexible for active software engineering tasks such as debugging, refactoring, writing tests, and creating pull requests. Their strength is the interactive agentic loop: gathering context, taking actions, observing results, and adapting. The RAG-Scientific-Code-Agent, on the other hand, is optimized for repeated question answering over a relatively stable scientific codebase. Its strength is controlled, reproducible, and domain-specific retrieval. It is therefore better suited for explaining and navigating a prepared codebase, while Codex- or Claude-style agents are better suited for changing and verifying code during active development.

5 Evaluation

5.1 Experimental Setup

We evaluate answer accuracy, faithfulness, and development utility using 100 benchmark questions generated with GPT-5.4 (Codex) [17] after providing the model with the complete IPPL codebase. The questions span the eleven categories in Table 6; the complete set is provided in Section C.

GPT-5.5 [18] is used as an LLM judge to compare each generated answer with its reference answer. The judge assigns 1 point to a correct answer, 0.5 to a partially correct answer, and 0 to an incorrect answer, and records a rationale for answers that are not fully correct. LLM-based

judging provides a scalable evaluation mechanism but is known to exhibit systematic biases [19]. This automatic evaluation design is a potential source of bias, as discussed in Section 5.3.

Question category	Count	What the questions ask
algorithm	12	Algorithmic behavior of core numerical methods, such as FFT Poisson solvers, CG iteration, interpolation, FFTs, and particle-field operations.
api_usage	12	How to use IPPL APIs in code, for example creating fields, meshes, boundary conditions, particle classes, initialization/finalization, and solver setup.
boundary_and_halo	5	Boundary condition types, differences between boundary conditions, ghost-cell population, halo exchange, and absorbing boundaries.
build_and_install	5	Build requirements, third-party dependencies, CMake/GPU configuration, Kokkos setup, and cluster installation instructions.
class_responsibility	13	Responsibilities of important classes such as <code>BareField</code> , <code>Field</code> , <code>FieldLayout</code> , <code>ParticleBase</code> , solver classes, and layout classes.
data_flow	9	How data moves through the codebase, including particle-to-grid charge deposition, grid-to-particle field gathering, halo exchange, particle migration, and solver data flow.
definition_location	15	Where specific implementations live in the source tree, such as MPI setup, Kokkos initialization, FFT initialization, load balancing, and solver implementations.
file_purpose	13	The role of important source/header files, for example <code>Ipp1.h</code> , <code>Ipp1.cpp</code> , <code>FFT.hpp</code> , <code>BareField.h</code> , and mini-app files.
numerical_meaning	3	Physical or numerical interpretation of methods, such as what Poisson solvers compute, what FDTD fields mean, and tradeoffs between FFT and CG solvers.
parallelism_and_kokkos	9	How IPPL uses Kokkos, MPI, execution spaces, <code>Kokkos::View</code> , parallel loops, mixed precision, and CPU/GPU portability.
testing_and_workflow	4	Development workflow, naming conventions, profiling, math-function conventions, and SLURM job templates.
Total	100	Complete current evaluation benchmark.

Table 6: Categories in the current evaluation benchmark and the type of questions each category contains.

For evaluating different models the pipeline uses one fixed model setup for the "explanation generation" and chunk embeddings.

Pipeline role	Model used in latest SSH evaluation
Embedding	nomic-embed-text
Chunk explanation model	qwen2.5-coder:14b
File-level model	qwen2.5-coder:32b-instruct-q4_K_M
Module-level model	qwen2.5-coder:32b-instruct-q4_K_M
Call-chain model	qwen2.5-coder:32b-instruct-q4_K_M

Table 7: Models used in the latest SSH evaluation run on the Gwendolen partition.

The only varying component is the answering language model, which receives the retrieved context, code, metadata, and previous LLM generations. The evaluation includes models from the Qwen2.5 [20], Qwen3.5 [21], and Gemma 4 [22] families. The tested configurations are listed below.

Model	Parameters	Context Window	Memory Sizes
Qwen2.5-coder-4_K_M	7B	32K	~7 GB
Qwen2.5-coder-4_K_M	14B	32K	~12 GB
Qwen2.5-coder-4_K_M	32B	32K	~24 GB
Qwen3.5	32B	256K	~28 GB
Qwen3.5	9B	256K	~9 GB
Gemma4	31B	256K	~24 GB
Gemma4	12B	256K	~11 GB

Table 8: Retrieval Answering models on which we evaluate the 100 Benchmark questions. Memory sizes are approximate VRAM requirements for the individual model.

In order to test retrieval quality directly the benchmark questions are also tested against a different architecture, for this the evaluation ran over the claude code [1] retrieval architecture with some of the same models used before in the test run with the RAG architecture.

Model	Parameters	Context Window	Memory Sizes
Claude Code (Qwen2.5-coder-4_K_M)	7B	32K	~7 GB
Claude Code (Qwen2.5-coder-4_K_M)	32B	32K	~24 GB
Claude Code (Gemma4)	12B	256K	~11 GB

Table 9: Models used in comparison for the Claude Code (Anthropic) [1] architecture on the benchmark questions for comparison to our architecture. The pipeline here uses the Claude Code architecture to find the answer inside the ippl codebase only using the specified smaller model.

5.2 Results

Evaluating the different models based on the explained setup in the previous section, showed interesting results. The older, well-established Qwen2.5 models underperformed with a range from 0.665 to 0.720. The Gemma4 models were also ranked among these values; however, the Qwen3.5 model is the clear winner out of the three. What was expected and can be clearly seen in the resulting data is that newer models or higher parameter counts tend to increase latency. Note that all latency was measured on the merlin6 ssh.

Model	Parameters	Correct [1]	Partial [0.5]	Incorrect [0]	Mean Latency	Average Score
Qwen2.5	7B	59	26	15	3.54s	0.720 ± 0.037
Qwen2.5	14B	60	17	23	6.95s	0.685 ± 0.042
Qwen2.5	32B	55	23	22	14.13s	0.665 ± 0.041
Qwen3.5	9B	70	19	11	18.14s	0.795 ± 0.034
Qwen3.5	32B	65	23	12	30.91s	0.765 ± 0.035
Gemma4	12B	56	23	21	38.72s	0.675 ± 0.040
Gemma4	31B	63	16	21	41.00s	0.710 ± 0.041

Table 10: Main results for 7 different models, based on 100 benchmark questions. Values after $\pm$ denote the standard error of the mean over $n = 100$ questions. Evaluated on Merlin6 via SSH by running `job_eval.sh`.

5.2.1 Claude Code Comparison Results

The results of a frontier retrieval architecture such as Claude Code [1] resulted in a smaller average score than for the Rag-Scientific code agent implemented in this project. The two instruct models of Qwen2.5-q4_K_M showed a score of 0.520 ± 0.038 for the smaller 7B model and an average score of 0.560 ± 0.036 for the 32B model. The Agentic loop alongside the Gemma4 model scored higher with an average score of 0.665 ± 0.040 .

Metric Parameters	Claude Code		
	Qwen2.5-q4_K_M 7B	Qwen2.5-q4_K_M 32B	Gemma4 12B
Correct	30	33	53
Partial	44	46	27
Incorrect	26	21	20
Mean Latency	6.2s	40.8s	92.4s
Average Score	0.520 ± 0.038	0.560 ± 0.036	0.665 ± 0.040

Table 11: Comparison results using the Claude Code [1] pipeline with Qwen2.5 and Gemma4 [22] models for comparison with the RAG pipeline.

What is notable when looking at the results is that for both instruct models most answers are only partially correct, with 44/100 questions 'Partial' for the 7B model and 46/100 for the 32B model. Whereas the Gemma4 model stays around the same amount of 20/100 'Incorrect' answers as the other two, but only marks 27/100 answers as 'Partial'.

5.2.2 Category Breakdown

Based on the seven evaluations we generated for our pipeline, the strongest category overall is API Usage. Across all models, it had 69 correct, 10 partial, and only 5 incorrect out of 84 total answers, giving an aggregate average score of 0.881. This means the models were usually good when the question asked how an API, class, method, or usage pattern works.

The aggregated breakdown across all evaluated models shows:

Category	Total answers	Correct	Partial	Incorrect	Avg. score
API Usage	84	69	10	5	0.881 ± 0.030
Definition Location	105	81	2	22	0.781 ± 0.040
Class Responsibility	91	63	13	15	0.764 ± 0.039
File Purpose	91	54	28	9	0.747 ± 0.035
Boundary And Halo	35	20	12	3	0.743 ± 0.056
Data Flow	63	38	17	8	0.738 ± 0.050
Algorithm	84	39	33	12	0.661 ± 0.039
Parallelism And Kokkos	63	27	23	13	0.611 ± 0.049
Testing And Workflow	28	16	2	10	0.607 ± 0.090
Numerical Meaning	21	11	1	9	0.548 ± 0.109
Build And Install	35	10	6	19	0.371 ± 0.073

Table 12: Aggregated category-level performance across all evaluated models. Values after $\pm$ denote the standard error of the mean.

The categories that worked well were API Usage, Definition Location, Class Responsibility, File Purpose, Boundary And Halo, and Data Flow. These all scored around 0.74 or higher, meaning the models often found the right file, class, function, or conceptual role. For example, Definition Location had 81 correct out of 105, and File Purpose had only 9 incorrect out of 91, even though many answers were partial.

The weakest category was Build And Install, with only 10 correct out of 35 and 19 incorrect, for an average score of 0.371. That suggests the models struggled with setup, installation, CMake/build instructions, and workflow details. Numerical Meaning was also weak, with 11 correct, 1 partial, and 9 incorrect out of 21, average 0.548, although this category has fewer questions. Parallelism And Kokkos and Testing And Workflow were also mixed, both around 0.61, likely because those questions require more cross-file reasoning and exact understanding of project conventions rather than simply locating a symbol.

Category	Questions	Correct	Partial	Incorrect	Avg. Score
File Purpose	13	9	4	0	0.846 ± 0.067
Definition Location	15	13	0	2	0.867 ± 0.091
Class Responsibility	13	9	2	2	0.769 ± 0.108
Algorithm	12	8	3	1	0.792 ± 0.096
Data Flow	9	5	3	1	0.722 ± 0.121
API Usage	12	11	1	0	0.958 ± 0.042
Parallelism and Kokkos	9	6	2	1	0.778 ± 0.121
Boundary and Halo	5	4	1	0	0.900 ± 0.100
Numerical Meaning	3	2	0	1	0.667 ± 0.333
Build and Install	5	1	2	2	0.400 ± 0.187
Testing and Workflow	4	2	1	1	0.625 ± 0.240

Table 13: Category breakdown for the Qwen3.5 9B model. Values after $\pm$ denote the standard error of the mean.

5.3 Discussion

A key observation in Table 10 is the decrease in accuracy when increasing model size. It is clearly visible for the Qwen2.5-coder model, where the 32B model performs with an average score of 0.055

lower than its smaller 7B model. Notable to see is, that this is no exception, the Qwen3.5 32B model is 0.03 lower than its 9B version. The only model which doesn't follow this rule is the gemma4 model in which an increase in Average Score of 0.035 can be seen when increasing the parameter size from 12B to 31B.

The reason for this phenomenon can vary and is not clear. What is clear is that the benchmark rewards short, grounded, repository-specific answers. Large models often give broader answers. If the model includes irrelevant details, misses the exact source location, or wanders from the question, the score drops.

What can be accounted for scoring accuracy are the general generation settings, same prompt, temperature, context length, and max tokens do not affect every model equally. A setting that works well for Qwen3.5 9B may be suboptimal for Gemma4 31B or Qwen2.5 32B model.

Generally the results in Table 10 show that model family matters more than size, a 9B model from a newer or better-aligned family can beat a 31B model from another family. In the results Qwen3.5 9B scored higher than Gema4 31B, likely because it was better suited to code/RAG-style question answering.

This is also again underlined when comparing Qwen3.5 9B with the 32B model which only differences 0.03 in the benefit of the 9B model. When comparing the 9B model to other models of similar size, the pattern holds: within the smaller size class, the newer Qwen3.5 family clearly outperforms the older Qwen2.5-coder generation, confirming that architectural and alignment improvements outweigh raw scale for this task.

A second, more important comparison concerns the retrieval architecture itself. Table 11 shows that the same models perform substantially worse when embedded in the Claude Code retrieval pipeline than in our dedicated RAG pipeline. The Qwen2.5 7B model drops from an average score of 0.720 in our pipeline to 0.520 under Claude Code. This is a striking result: the agentic retrieval loop, which is designed around very large frontier models, does not transfer well to small local models. Without a strong model to drive iterative file inspection and tool use, the agentic approach appears to retrieve noisier or less targeted context, whereas our offline-prepared retrieval context is already condensed into grounded, repository-specific explanations that a small model can use directly.

This supports the central hypothesis of the project: when the answering model is small, the quality of the prepared retrieval context matters more than the sophistication of the retrieval-time agent loop. Front-loading code understanding into the ingestion stage effectively compensates for the limited reasoning capacity of small models.

Two caveats should be noted when interpreting these results. First, the benchmark answers and the calibration baseline were generated by OpenAI models (GPT-5.4 for question generation, GPT-5.5 for scoring). An LLM judge may carry subtle stylistic preferences—for example, favouring concise, structured answers—which could systematically advantage or disadvantage certain answering models independently of factual correctness. Second, the benchmark consists of 100 questions over a single codebase (IPPL), so the absolute scores should be read as indicative rather than definitive. A larger multi-codebase benchmark and, ideally, partial human validation of the LLM judgments would strengthen these conclusions.

6 Conclusion

This study investigated whether a useful coding agent can be built around small, locally executed open-source models. The implemented RAG scientific-code agent demonstrates that this is feasible: by separating an expensive offline ingestion stage from a lightweight online answering stage, the

system enables small models to produce grounded, repository-specific answers on the IPPL codebase. The strongest configuration, a 9B model, achieved an average score of 0.795 ± 0.034 on a 100-question benchmark, outperforming larger models within the same pipeline as well as the same models embedded in an agentic Claude Code retrieval architecture.

Two findings stand out. First, for this task model family and retrieval quality matter more than parameter count—newer, better-aligned model families consistently outperformed larger but older ones. Second, the offline-prepared retrieval context appears to be precisely what allows small models to remain competitive, since it condenses code understanding into explanations and structural metadata that a small model can use without extensive reasoning of its own.

It is worth emphasizing what the system is and is not. The current architecture is based solely on semantically explaining a codebase rather than acting as an all-round code agent; it does not edit files, run tests, or iteratively verify its own output. The result is therefore best understood as a privacy-preserving “code teacher” that runs locally and free of charge, allowing anyone to explore how a scientific codebase works while keeping their data entirely on their own infrastructure. For active software engineering tasks, additional tooling would be required.

The central trade-off of the design is its greatest strength and its greatest weakness. Front-loading code understanding into the ingestion stage reduces inference cost and lets small models perform well, but any change to the codebase requires re-running the costly ingestion pipeline. This makes the agent best suited to relatively stable codebases that are queried repeatedly, such as those found in educational settings—universities, schools, or corporate onboarding—where new students or employees can rely on a safe, local, and free tool to learn how a large scientific codebase is structured and how it works.

6.1 Future Work

Although the project is already substantial, several parts of the pipeline remain to be improved or tested. Any such improvement should be evaluated carefully and in context, since changes made without proper evaluation could just as easily degrade the pipeline rather than improve it.

Starting at the beginning of the pipeline, the embedding stage, one improvement would be clearer formatting and symbolisation of code blocks, for example marking them explicitly with backticks:

```
1  -- CHUNK START --
2      Document Section: User Authentication Handler
3      Semantic Context: This function validates the incoming JSON payload.
4      Code:
5      ```python
6          def verify_age(payload):
7              age = payload.get("age", 0)
8              return isinstance(age, int) and age >= 18
9      ```
10 -- CHUNK END --
```

This, along with other experiments in chunk formatting, could improve both retrieval and context quality, since chunk formatting plays an important role in how well an LLM understands the input.

A further set of improvements involves swapping individual software components in the pipeline, such as the embedder, the retriever, or the vector store, as well as adding a pre-built re-ranking library. For re-ranking, BAAI/bge-reranker-v2-m3 or BAAI/bge-reranker-large would be candidates for evaluation. Cross-encoder rerankers add computation to retrieval, but they may recover text chunks that describe a code concept accurately despite receiving a low initial vector-similarity score.

Finally, the agent could be extended with external tooling such as `opencode` to act as an implementation helper rather than purely a learning tool. This would likely require a revision of the codebase first, so it should be treated as a longer-term goal. In the nearer term, the higher priority is constant evaluation while experimenting with alternative frameworks, retrievers, models, and embedders.

Data and Code Availability

The implementation, evaluation material, and instructions needed to reproduce the reported experiments are available at <https://github.com/nobileaaron/rag-scientific-code-agent>.

Acknowledgements

The authors thank the contributors to the IPPL codebase and the computing staff who supported the evaluation runs.

References

- [1] Anthropic. Claude code. <https://www.anthropic.com/claude-code>, 2025. Accessed: 2026-07-09.
- [2] OpenAI. Openai codex. <https://openai.com/codex>, 2025. Accessed: 2026-07-09.
- [3] IPPL Framework Developers. IPPL: A C++ library to develop performance portable code for fully Eulerian, Lagrangian or hybrid Eulerian-Lagrangian methods. <https://github.com/IPPL-framework/ippl>, 2024. Accessed: 2026-07-22.
- [4] Sriramkrishnan Muralikrishnan, Matthias Frey, Alessandro Vinciguerra, Michael Ligotino, Antoine J. Cerfon, Miroslav Stoyanov, Rahulkumar Gayatri, and Andreas Adelmann. Scaling and performance portability of the particle-in-cell scheme for plasma physics applications through mini-apps targeting exascale architectures. In *Proceedings of the 2024 SIAM Conference on Parallel Processing for Scientific Computing*, pages 26–38. Society for Industrial and Applied Mathematics, 2024.
- [5] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jiang-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2471–2484. Association for Computational Linguistics, 2023.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra,

- Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
 - [8] Haskell B. Curry. The method of steepest descent for non-linear minimization problems. *Quarterly of Applied Mathematics*, 2(3):258–261, 1944.
 - [9] Google. Gemini code assist. <https://developers.google.com/gemini-code-assist>, 2025. Accessed: 2026-07-20.
 - [10] Anysphere. Cursor: The AI code editor. <https://cursor.com>, 2025. Accessed: 2026-07-20.
 - [11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
 - [12] H. Carter Edwards, Christian R. Trott, and Daniel Sunderland. Kokkos: Enabling manycore performance portability through polymorphic memory access patterns. *Journal of Parallel and Distributed Computing*, 74(12):3202–3216, 2014.
 - [13] Alan Ayala, Stanimire Tomov, Azzam Haidar, Piotr Luszczek, and Jack Dongarra. heFFTe: Highly efficient FFT for exascale. In *Computational Science–ICCS 2020*, pages 262–275. Springer, 2020.
 - [14] Tree-sitter Contributors. Tree-sitter: An incremental parsing system for programming tools. Software repository, 2026. Accessed: 2026-09-04.
 - [15] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
 - [16] Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed: Training a reproducible long context text embedder. *arXiv preprint arXiv:2402.01613*, 2024.
 - [17] OpenAI. Introducing GPT-5.4. Product and technical release, 2026. Accessed: 2026-09-04.
 - [18] OpenAI. Introducing GPT-5.5. Product and technical release, 2026. Accessed: 2026-09-04.
 - [19] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
 - [20] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
 - [21] Qwen Team. Qwen3.5. Technical release and model documentation, 2026. Accessed: 2026-09-04.
 - [22] Gemma Team. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*, 2026.

A Evaluation Protocol

```
1     You are evaluating the currently selected Claude Code model on the IPPL
    benchmark.
2
3 Task:
4 Read 'docs/evaluations/eval_questions_v2.json', answer every question using only
    evidence from this repository, especially 'data/raw/ippl' and repository
    documentation, and save the results as a JSON file under 'docs/evaluations/
    answers/'.
5
6 Important requirements:
7 - Do not modify source code or configuration files.
8 - Create 'docs/evaluations/answers/' if it does not exist.
9 - Use the environment variable 'CLAUDE_EVAL_MODEL_LABEL' as the run label. If it
    is not set, use 'unknown_model'.
10 - Save to 'docs/evaluations/answers/claude_code_<run_label>_<UTC timestamp>.json'.
11 - Write progress incrementally after each answered question with '"run_complete":
    false', so partial progress is preserved if the run stops.
12 - At the end, rewrite the same file with '"run_complete": true'.
13 - Use this output schema:
14
15 {
16   "schema": "claude-code-direct-eval/v1",
17   "run_complete": false,
18   "run_label": "...",
19   "run_started_at_utc": "...",
20   "run_finished_at_utc": null,
21   "questions_source": "docs/evaluations/eval_questions_v2.json",
22   "answering_instructions": "...",
23   "answers": [
24     {
25       "id": "...",
26       "category": "...",
27       "question": "...",
28       "answer": "...",
29       "evidence_files": [...],
30       "error": null,
31       "latency_seconds": 0.0
32     }
33   ]
34 }
35
36 Answering rules:
37 - For each question, give a concise technical answer.
38 - Ground the answer in concrete files, classes, functions, or documentation
    sections when possible.
39 - Include relevant evidence file paths in 'evidence_files'.
40 - If the repository evidence is insufficient, say so explicitly in the answer
    instead of guessing.
41 - Do not use external web search.
42 - Do not include long copied code blocks unless necessary.
43 - Keep answers comparable across runs: use the same style and level of detail for
    every model.
44
45 Implementation:
46 Use shell/Python as needed to parse the JSON and write the output file, but
    generate the actual answer content yourself from repository inspection. Proceed
```

```
through all questions without asking me for confirmation.
```

B Prompt Templates

The complete system and user prompts for chunk explanation, file- and module-level summarization, call-chain summarization, and retrieval-based answering are distributed with the reproducibility artifacts in `appendices/prompt_templates_overview.txt` and in the public code repository. They are omitted here because several templates are long machine inputs whose verbatim inclusion does not aid interpretation of the method.

C Evaluation Questions

The complete benchmark is distributed as `appendices/eval_questions_v2.txt` and with the public repository. It contains 100 questions in the eleven categories summarized in Table 6. Providing the artifact separately keeps the manuscript concise while preserving the exact benchmark wording.